Employee ID Management using BST (C++)

```cpp
#include <iostream>
#include <string>
#include <limits>
using namespace std;

struct Employee {
    int id;
    string name;
    Employee* left;
    Employee* right;

    Employee(int empId, string empName) {
        id = empId;
        name = empName;
        left = nullptr;
        right = nullptr;
    }
};

// Insert employee into the BST
Employee* insertEmployee(Employee* root,
                         int id, string name) {
    if (root == nullptr) {
        cout << "Employee added successfully.\n";
        return new Employee(id, name);
    }

    if (id < root->id) {
        root->left = insertEmployee(root->left,
                                    id, name);
    }
    else if (id > root->id) {
        root->right = insertEmployee(root->right,
                                     id, name);
    }
    else {
        cout << "Duplicate employee ID not allowed.\n";
    }

    return root;
}

// Search employee and count comparisons
void searchEmployee(Employee* root, int id) {
    int comparisons = 0;
    Employee* current = root;

    while (current != nullptr) {
        comparisons++;

        if (id == current->id) {
            cout << "\nEmployee found!\n";
            cout << "Employee ID: "
                 << current->id << "\n";
            cout << "Employee Name: "
                 << current->name << "\n";
            cout << "Comparisons: "
                 << comparisons << "\n";
            return;
        }

        if (id < current->id) {
            current = current->left;
        } else {
            current = current->right;
        }
    }

```

```
    cout << "\nEmployee not found.\n";
    cout << "Comparisons: "
         << comparisons << "\n";
}

// Display employees in ascending ID order
void displayEmployees(Employee* root) {
    if (root == nullptr) {
        return;
    }

    displayEmployees(root->left);

    cout << "ID: " << root->id
         << " | Name: " << root->name << "\n";

    displayEmployees(root->right);
}

int main() {
    Employee* root = nullptr;
    int choice, id;
    string name;

    do {
        cout << "\n===== Employee ID Management =====\n";
        cout << "1. Insert Employee\n";
        cout << "2. Search Employee\n";
        cout << "3. Display Employees in Ascending Order\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";

        if (!(cin >> choice)) {
            cout << "Invalid input. Exiting.\n";
            break;
        }

        switch (choice) {
        case 1:
            cout << "Enter Employee ID: ";
            if (!(cin >> id) || id <= 0) {
                cout << "Enter a positive integer ID.\n";
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            }

            cin.ignore(numeric_limits<streamsize>::max(), '\n');

            cout << "Enter Employee Name: ";
            getline(cin, name);

            if (name.empty()) {
                cout << "Name cannot be empty.\n";
                break;
            }

            root = insertEmployee(root, id, name);
            break;

        case 2:
            cout << "Enter Employee ID to search: ";
            if (!(cin >> id) || id <= 0) {
                cout << "Enter a positive integer ID.\n";
                cin.clear();
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                break;
            }
```

```
137
138             searchEmployee(root, id);
139             break;
140
141         case 3:
142             if (root == nullptr) {
143                 cout << "No employee records available.\n";
144             } else {
145                 cout << "\nEmployees in Ascending ID Order:\n";
146                 displayEmployees(root);
147             }
148             break;
149
150         case 4:
151             cout << "Exiting program.\n";
152             break;
153
154         default:
155             cout << "Invalid choice. Try again.\n";
156         }
157
158     } while (choice != 4);
159
160     return 0;
161 }
162
```